

CCF2026开源创新大赛

Mooncake赛题2初赛技术报告

Mooncake Store碎片感知分配优化

Fragmentation-Aware Allocation for Mooncake Store

参赛方向	赛题2:优化Mooncake Store吞吐性能、高可用功能和可扩展性,优化SGLang HiCache+Mooncake Store性能
作品名称	Mooncake Store碎片感知分配策略
参赛仓库	https://github.com/Lorry1024/ccf2026-mooncake-fragmentation-aware
上游PR	https://github.com/kvcache-ai/Mooncake/pull/2797
PR头提交	0123fa1 Fix fragmentation-aware allocation test setup
报告日期	2026年7月9日

本报告与仓库中的patch、复现实验、验证日志和展示PPT共同构成初赛提交材料。

摘要

Mooncake Store是Mooncake体系中支撑KVCache对象存储和复用的关键组件。现有`free_ratio_first`策略根据segment总空闲比例进行排序，能够改善整体空间利用均衡，但在混合大小KVCache对象和长时间运行的场景下，总空闲空间可能被切分为多个小空洞，无法真实反映当前大对象是否可以被连续放入。本文提出并实现`fragmentation_aware`策略，在保持默认行为不变和有界候选采样不变的前提下，将最大连续空闲区域、连续性比例和当前请求可容纳性纳入排序逻辑，使Master优先选择能够直接满足请求的segment。

本作品已形成Mooncake上游draft PR，当前PR头提交0123fa1已通过GitHub Actions，结果为26个检查成功、1个检查跳过。确定性专题仿真显示，在6个大对象请求上，`free_ratio_first`的首选segment可直接容纳次数为0/6，而`fragmentation_aware`为6/6；fallback尝试次数从11降至0；平均候选segment数量保持5.00；本地单次决策耗时从121.00ns增加到138.93ns，额外开销约17.93ns。该结果说明，新策略能够以很小的决策成本提升碎片化场景下的分配稳定性。

关键词： Mooncake Store； KVCache； 碎片感知； 连续空闲区域； 分配策略； SGLang HiCache

模块完成情况

表 1: 初赛阶段模块完成情况

模块	完成情况
问题分析	明确 <code>free_ratio_first</code> 在碎片化大对象分配中的误判模式，形成最小复现和专题仿真。
策略设计	设计 <code>fragmentation_aware</code> 排序规则，优先考虑 <code>can_fit</code> 、连续性评分和最大连续空闲区域。
代码实现	在Mooncake Store中新增策略类型、策略类、配置解析、启动参数和benchmark矩阵。
测试验证	新增碎片化单元测试，保留 <code>preferred segment</code> 语义，PR通过上游GitHub Actions。
材料整理	完成中文Markdown材料、LaTeX技术报告、展示PPT、复现实验源码和提交包。
待完成项	5分钟展示视频尚未录制；真实RDMA和SGLang HiCache端到端benchmark作为后续工作。

目录

1	赛题理解与项目定位	1
2	原有机制与问题分析	1
2.1	Mooncake Store原有分配策略	1
2.2	总空闲比例的误判	1
3	碎片感知分配策略设计	2
3.1	设计原则	2
3.2	评分模型	2
4	实现路径	3
4.1	参数启用方式	4
4.2	测试修复过程	5
5	验证与效果	5
5.1	CI验证	5
5.2	专题仿真指标	5
5.3	结果解释	5
6	与赛题要求的对应	6
7	使用与复现	7
7.1	启用策略	7
7.2	应用补丁	7
7.3	回滚补丁	7
7.4	复现实验	7
8	边界与后续工作	8
9	结论	8
A	提交材料清单	8
B	参考链接	9

1 赛题理解与项目定位

Mooncake赛题2要求围绕Mooncake Store吞吐性能、高可用功能、可扩展性以及SGLang HiCache+Mooncake Store性能开展优化。该要求并不只局限于网络传输或外层推理框架，也包含Store后端内部影响吞吐稳定性和资源利用率的关键路径。对KVCache场景而言，Store中对象的分配位置会影响一次写入是否能顺利落地，也会影响后续fallback、重试、淘汰和副本放置压力。

本项目选择的切入点是Mooncake Store分配策略层。该方向具有三个特点：第一，它位于真实Mooncake Store代码路径中，能够通过PR形式提交给上游；第二，它对SGLang HiCache等前端具有后端相关性，因为这些前端将KVCache对象交给Store时需要依赖Store的放置决策；第三，它可以在不重写协议、不引入重依赖、不改变默认行为的条件下完成可验证优化。

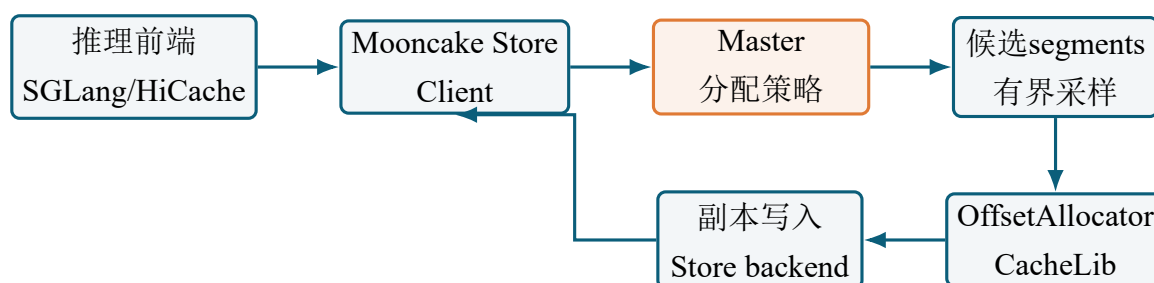


图 1: 本项目在Mooncake Store路径中的位置

2 原有机制与问题分析

2.1 Mooncake Store原有分配策略

Mooncake Store已有多种segment分配策略。默认random策略开销最低，适合简单场景；free_ratio_first策略按照总空闲比例排序，有利于改善segment之间的负载和空间均衡；其他策略则面向SSD、CXL或local-first等特定场景。对本项目而言，关键对比对象是free_ratio_first，因为它已经尝试利用空闲空间信息，但仍停留在“总量”视角。

2.2 总空闲比例的误判

在长期运行的KVCache Store中，对象大小可能从几MiB到几十MiB不等。对象释放后形成的空闲区域并不一定连续，导致一个segment拥有较高总空闲比例，却无法为当前请求提供足够大的连续空间。此时只看总空闲比例会产生错误排序。

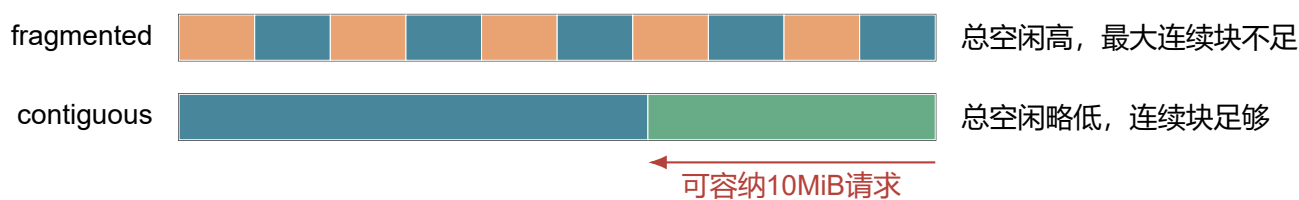


图 2: 总空闲空间与最大连续空闲区域可能不一致

最小复现数据如表2所示。`free_ratio_first`会优先选择总空闲空间为16MiB的`fragmented`，但它的最大连续空闲区域只有8MiB，无法容纳10MiB请求；`fragmentation_aware`会选择总空闲空间为12MiB但连续区域为12MiB的`contiguous`。

表 2: 最小碎片化复现场景

segment	总容量	总空闲	最大连续空闲	10MiB请求
fragmented	32MiB	16MiB	8MiB	不能容纳
contiguous	32MiB	12MiB	12MiB	可以容纳

3 碎片感知分配策略设计

3.1 设计原则

`fragmentation_aware`遵循四个原则：

1. 保持默认行为不变。用户不显式指定新策略时，Mooncake Store仍使用原有默认策略。
2. 保持有界采样。策略只对采样候选评分，不进行全局segment扫描。
3. 优先满足当前请求。排序时先判断最大连续空闲区域是否能够容纳当前对象。
4. 保留原有语义。`preferred segment`、`excluded segment`和`best-effort replica`行为不能被破坏。

3.2 评分模型

对每个候选segment，策略读取或计算以下信息：

$$total_free = \sum(capacity - used) \quad (1)$$

$$largest_free_region = \max(getLargestFreeRegion()) \quad (2)$$

$$free_ratio = \frac{total_free}{total_capacity} \quad (3)$$

$$contiguity_ratio = \frac{largest_free_region}{total_free} \quad (4)$$

$$score = 0.70 \times contiguity_ratio + 0.30 \times free_ratio \quad (5)$$

排序规则为：先比较`can_fit`，再比较`score`，再比较`largest_free_region`，再比较`free_ratio`，最后使用稳定`tie-break`保证排序结果可复现。

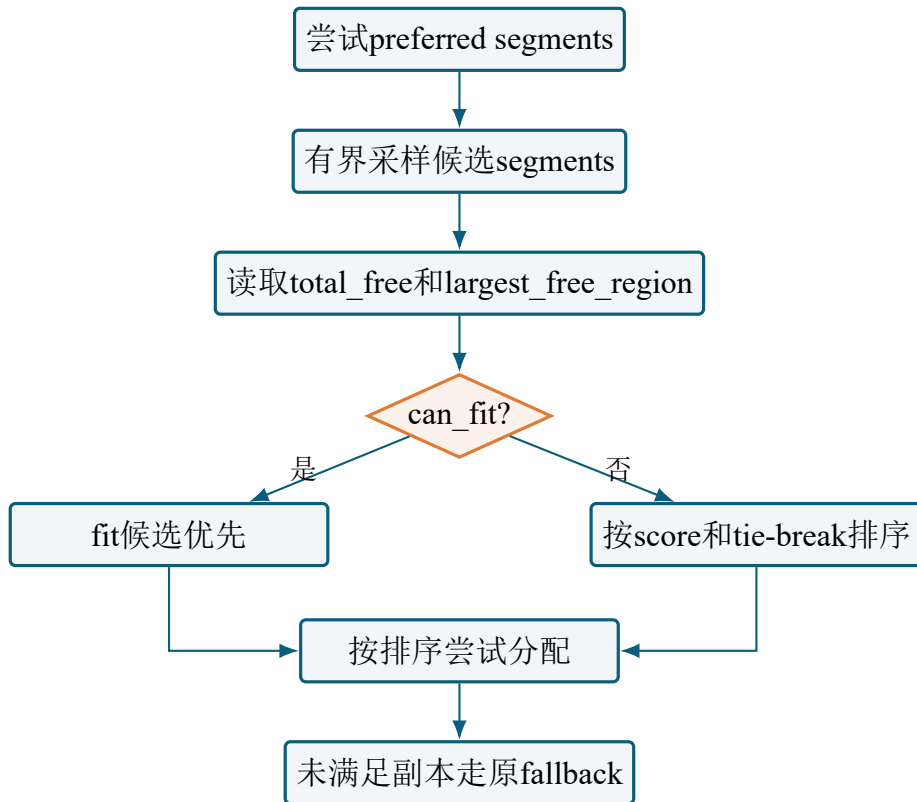


图 3: fragmentation_aware策略流程

4 实现路径

当前PR补丁涉及9个文件，覆盖策略实现、配置接入、测试、benchmark、文档和CI辅助步骤。表3列出主要改动。

表 3: 关键代码改动

文件	作用
mooncake-store/include allocation_strategy.h	新增FragmentationAwareAllocationStrategy, 实现候选评分和排序。
mooncake-store/include types.h	新增AllocationStrategyType::FRAGMENTATION_AWARE。
mooncake-store/include master_config.h	解析allocation_strategy=fragmentation_aware。
mooncake-store/src master.cpp	更新启动参数帮助信息。
mooncake-store/tests allocation_strategy_test.cpp	增加碎片化定向测试和preferred segment测试。
mooncake-store/benchmarks allocation_strategy_bench.cpp	将新策略加入benchmark矩阵。
docs/source/design mooncake-store.md	增加策略说明和使用建议。
docs/source/deployment mooncake-store-deployment-guide.md	增加部署参数说明。
.github/workflows ci.yml	在Go store binding集成测试前释放runner磁盘空间, 避免无关失败。

4.1 参数启用方式

新策略通过Master启动参数显式启用:

```
mooncake_master --allocation_strategy=fragmentation_aware
```

由于默认策略保持为random, 因此老用户不受影响。出现异常时只需要切回random或free_ratio_first即可回滚策略行为。

4.2 测试修复过程

PR创建后经历了三类CI问题并逐步修复。第一次失败来自代码格式检查，后续通过clang-format修复。第二次失败来自GitHub runner磁盘不足，Go store binding集成测试链接阶段出现空间不足，后续在对应job前增加磁盘清理。第三次失败来自碎片化测试构造不足，释放块与尾部空闲区域发生合并，导致断言不稳定，后续通过调整构造为5个8MiB块并释放内部块修复。最终PR头提交0123fa1通过全部必需检查。

5 验证与效果

5.1 CI验证

Mooncake上游draft PR状态如表4所示。

表 4: PR CI状态

项目	内容
PR链接	https://github.com/kvcache-ai/Mooncake/pull/2797
头提交	0123fa1 Fix fragmentation-aware allocation test setup
CI结果	26个检查成功，1个检查跳过
结论	上游构建、格式检查、文档构建和相关测试已通过。

5.2 专题仿真指标

专题仿真使用5个segment和6个大对象请求，模拟高总空闲但碎片化、低总空闲但连续空间充足两类segment并存的Store状态。结果如表5所示。

5.3 结果解释

该结果有三个含义。第一，fragmentation_aware显著改善首选segment质量，在所有测试请求上都能首先选中可容纳segment。第二，平均候选segment数量没有增加，说明改进来自排序信号变化，而不是扩大搜索范围。第三，单次决策新增开销约17.93ns，在本地CPU仿真中处于很小量级，符合“用少量元数据计算换取更稳定分配路径”的设计目标。

表 5: 专题对齐仿真结果

指标	free_ratio_first	fragmentation_aware
大对象首选segment可直接容纳	0/6	6/6
最终可容纳	6/6	6/6
fallback尝试次数	11	0
平均候选segment数量	5.00	5.00
单次决策耗时	121.00ns	138.93ns
新增决策开销	不适用	17.93ns

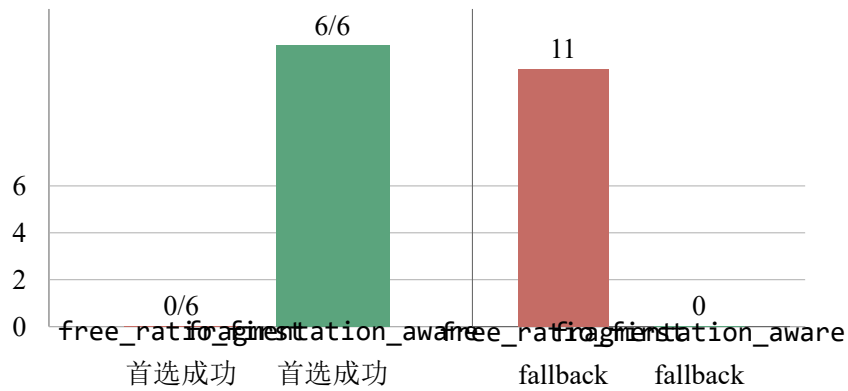


图 4: 首选成功次数与fallback尝试次数对比

6 与赛题要求的对应

表 6: 赛题要求映射

赛题关注点	本项目实现	当前证据
Mooncake Store吞吐性能	减少碎片化导致的无效首选分配和fallback压力	专题仿真中fallback尝试次数从11降为0
可扩展性	保持有界候选采样，不做全局扫描	平均候选segment数量保持5.00
高可用语义	不改变副本best-effort路径，保留preferred/excluded语义	单元测试和策略设计说明

SGLang	Hi-	优化Store后端对象放置路径，为HiCache后端稳定性提供基础	方案定位和代码路径位于Mooncake Store
Cache+Store			
开源规范		以PR形式提交上游Mooncake,并保留patch和文档	PR#2797通过CI
可复现性		提供仿真源码、日志、技术报告和使用说明	repro/、logs/、EVALUATION.md

7 使用与复现

7.1 启用策略

```
mooncake_master --allocation_strategy=fragmentation_aware
```

7.2 应用补丁

```
git apply --check mooncake_fragmentation_aware_pr_2797_0123fa1.patch
git apply mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

7.3 回滚补丁

```
git apply -R mooncake_fragmentation_aware_pr_2797_0123fa1.patch
```

7.4 复现实验

```
g++ -std=c++17 -O2 repro/fragmentation_aware_sim.cpp -o
    fragmentation_aware_sim
./fragmentation_aware_sim

g++ -std=c++17 -O2 repro/fragmentation_aware_metrics.cpp -o
    fragmentation_aware_metrics
./fragmentation_aware_metrics

g++ -std=c++17 -O2 repro/topic_aligned_store_scalability_sim.cpp -o
    topic_aligned_store_scalability_sim
```

```
./topic_aligned_store_scalability_sim
```

8 边界与后续工作

本项目当前阶段已经完成代码实现、上游PR、CI验证、仿真评估和中文提交材料整理，但仍有边界需要明确：

- 当前不宣称已经完成真实RDMA硬件benchmark。
- 当前不宣称已经完成真实SGLang HiCache端到端吞吐压测。
- 当前不宣称已经重设计Mooncake高可用协议。
- 当前专题仿真验证的是分配策略层，不等同于完整生产集群压测。

后续如果进入决赛，建议补充三类工作：第一，在真实Mooncake Store环境中运行allocation_strategy_bench，统计吞吐、P50/P99、失败分配次数、fallback次数和内存利用率；第二，在具备RDMA条件的集群中验证跨节点场景；第三，将SGLang HiCache接入Mooncake Store后端，使用真实推理负载验证KVCache复用和后端IO表现。

9 结论

本文围绕Mooncake赛题2提出并实现Mooncake Store碎片感知分配策略。该策略针对free_ratio_first只看总空闲比例的不足，引入最大连续空闲区域和可容纳性排序，在不改变默认行为、不扩大候选规模、不破坏原有副本语义的前提下，提升碎片化场景下的大对象首选分配质量。当前PR已经通过上游GitHub Actions，确定性仿真显示首选成功次数从0/6提升到6/6，fallback尝试次数从11降到0。该工作是一个边界清晰、可复现、可回滚、适合继续扩展到真实Store benchmark和SGLang HiCache端到端测试的初赛成果。

A 提交材料清单

文件或目录	说明
README.md	项目入口说明
SUBMISSION.md	平台提交说明
DESIGN.md	设计文档

<code>technical_solution.md</code>	技术方案说明
<code>EVALUATION.md</code>	评估报告
<code>testing.md</code>	测试说明
<code>usage.md</code>	使用说明
<code>REVIEW_GUIDE.md</code>	评审阅读指南
<code>mooncake_fragmentation_aware</code>	当前PR补丁
<code>pr_2797_0123fa1.patch</code>	
<code>repro/</code>	复现实验源码
<code>logs/</code>	验证日志
<code>slides/</code>	初赛展示PPT
<code>report/</code>	本技术报告源码和PDF
<code>release/</code>	提交压缩包和校验文件

B 参考链接

- Mooncake上游仓库: [kvcache-ai/Mooncake](#)
- 本项目Mooncake PR: [kvcache-ai/Mooncake#2797](#)
- 比赛材料仓库: [Lorry1024/ccf2026-mooncake-fragmentation-aware](#)
- Mooncake fork分支: [Lorry1024/Mooncake:ccf-fragmentation-aware-allocation](#)